

Complete Technical Guide: Real-Time Height Measurement System

Computer Vision Project

Contents

1	Project Overview	2
2	System Requirements	2
3	Installation and Running Procedure	2
3.1	Step 1: Install Anaconda	2
3.2	Step 2: Create Environment	2
3.3	Step 3: Install Required Libraries	3
3.4	Step 4: Create Python File	3
3.5	Step 5: Camera Selection	3
3.6	Step 6: Run Program	3
4	Improved System Architecture	3
5	How the Height is Measured (Human Sketch)	4
5.1	Calibration Formula	5
5.2	Final Height Calculation	5
6	Project Screenshot	6
7	Complete Source Code	6
8	Conclusion	8

1 Project Overview

This document presents a complete implementation guide for a real-time human height measurement system using:

- Python
- OpenCV
- MediaPipe
- NumPy
- pyttsx3 (Text-to-Speech)

The system:

- Uses the laptop built-in webcam by default (camera index 0)
- Can also use an external USB camera (index 1 or higher)
- Calibrates using face height
- Detects full body pose landmarks
- Computes height in centimeters and feet/inches
- Announces the measured height

2 System Requirements

- Windows / macOS / Linux
- Anaconda installed
- Webcam (internal or USB external)
- Python 3.9–3.10 recommended

3 Installation and Running Procedure

3.1 Step 1: Install Anaconda

Download from:

<https://www.anaconda.com/products/distribution>

3.2 Step 2: Create Environment

```
conda create -n height_env python=3.10
conda activate height_env
```

3.3 Step 3: Install Required Libraries

```
pip install opencv-python
pip install mediapipe
pip install numpy
pip install pyttvx3
```

3.4 Step 4: Create Python File

Create:

`heightmmeasurement.py`

Copy the full source code from Section 7.

3.5 Step 5: Camera Selection

Default (Laptop Camera):

```
cap = cv.VideoCapture(0)
```

External USB Camera:

```
cap = cv.VideoCapture(1)
```

Camera Index Guide:

- 0 → Built-in laptop webcam
- 1 → First external camera
- 2 → Second external camera

3.6 Step 6: Run Program

```
python height_measurement.py
```

4 Improved System Architecture

The following diagram shows a clean hierarchical system architecture without overlapping nodes.

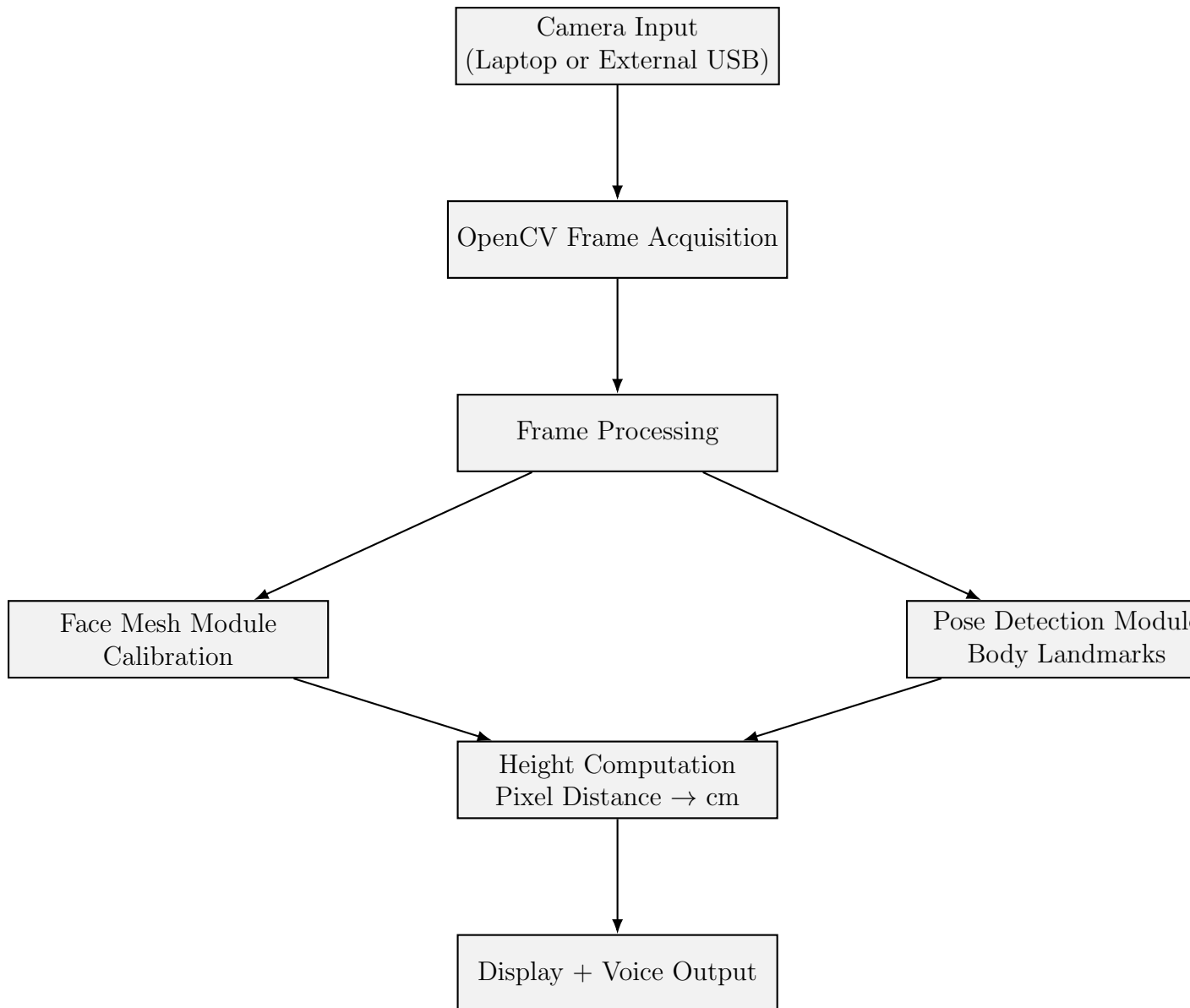


Figure 1: Improved Non-Overlapping System Architecture

5 How the Height is Measured (Human Sketch)

The algorithm measures the vertical pixel distance between:

- Nose landmark (top reference)
- Foot / ankle landmark (bottom reference)

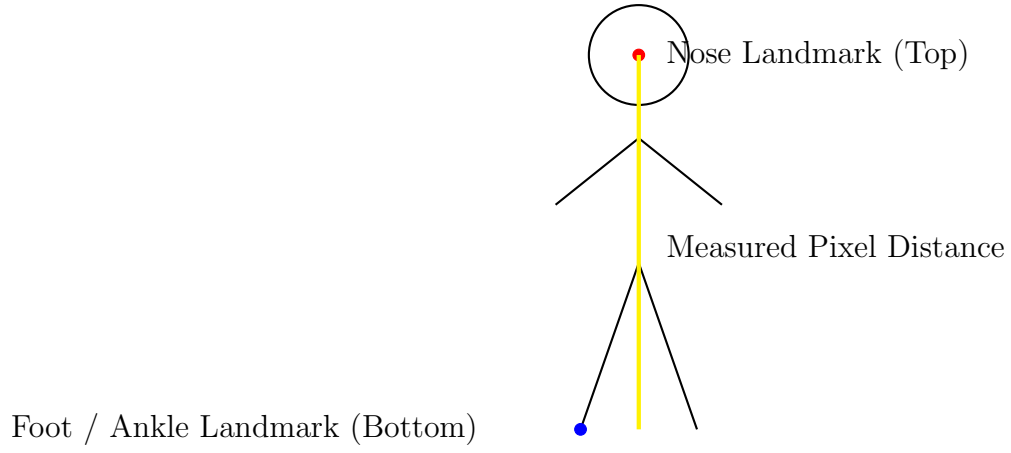


Figure 2: Height Measurement Principle

5.1 Calibration Formula

$$\text{Calibration Ratio} = \frac{20}{\text{Face Height (pixels)}}$$

5.2 Final Height Calculation

$$\text{Height (cm)} = (\text{Bottom}_y - \text{Nose}_y) \times \text{Calibration Ratio}$$

6 Project Screenshot

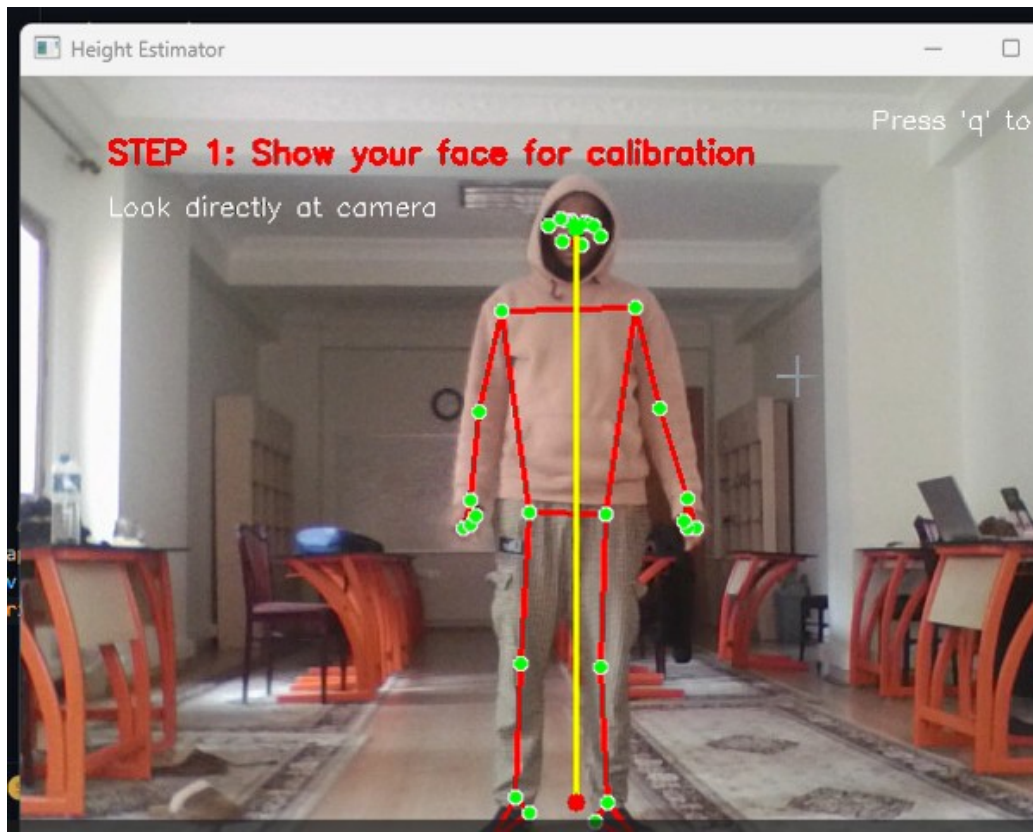


Figure 3: System Output Interface

7 Complete Source Code

```
# =====  
# HEIGHT MEASUREMENT SYSTEM  
# =====  
  
import cv2 as cv  
import mediapipe as mp  
import numpy as np  
import pyttsx3  
  
engine = pyttsx3.init()  
def speak(text):  
    engine.say(text)  
    engine.runAndWait()  
  
mp_pose = mp.solutions.pose  
mp_face_mesh = mp.solutions.face_mesh  
mp_draw = mp.solutions.drawing_utils  
  
pose = mp_pose.Pose(0.5,0.5)  
face_mesh = mp_face_mesh.FaceMesh(0.5,0.5)
```

```

# 0 = laptop cam, 1 = external cam
cap = cv.VideoCapture(0)

FACE_HEIGHT_CM = 20
calibration_ratio = None
measurements = []
current_height = 0
height_announced = False

while True:
    success, img = cap.read()
    if not success:
        break

    img = cv.flip(img, 1)
    h, w, _ = img.shape
    img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)

    face_results = face_mesh.process(img_rgb)
    pose_results = pose.process(img_rgb)

    if face_results.multi_face_landmarks and calibration_ratio is None:
        for face_landmarks in face_results.multi_face_landmarks:
            y_coords = [lm.y * h for lm in face_landmarks.landmark]
            face_h_px = max(y_coords) - min(y_coords)
            if face_h_px > 20:
                calibration_ratio = FACE_HEIGHT_CM / face_h_px
                speak("Calibration complete")

    if pose_results.pose_landmarks and calibration_ratio is not None:
        mp_draw.draw_landmarks(img,
                                pose_results.pose_landmarks,
                                mp_pose.POSE_CONNECTIONS)

        landmarks = pose_results.pose_landmarks.landmark
        nose_y = landmarks[0].y * h
        ankle_y = landmarks[28].y * h

        height_px = ankle_y - nose_y
        height_cm = height_px * calibration_ratio

        if 100 < height_cm < 250:
            measurements.append(height_cm)

            if len(measurements) > 10:
                measurements.pop(0)

```

```

        if len(measurements) >= 5:
            current_height = sum(measurements)/len(
                measurements)

            cv.putText(img,
                       f"{current_height:.1f} cm",
                       (50,100),
                       cv.FONT_HERSHEY_SIMPLEX,
                       2,
                       (0,255,255),
                       4)

            if not height_announced:
                speak(f"Your height is {current_height:.0f} cm")
                height_announced = True

    cv.imshow("Height Measurement", img)

    if cv.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv.destroyAllWindows()

```

8 Conclusion

This rewritten document includes:

- Clean non-overlapping architecture diagram
- Human sketch explaining measurement principle
- Mathematical explanation
- Camera configuration (internal and external)
- Full working source code

The system can be replicated easily by following the setup and execution steps described above.